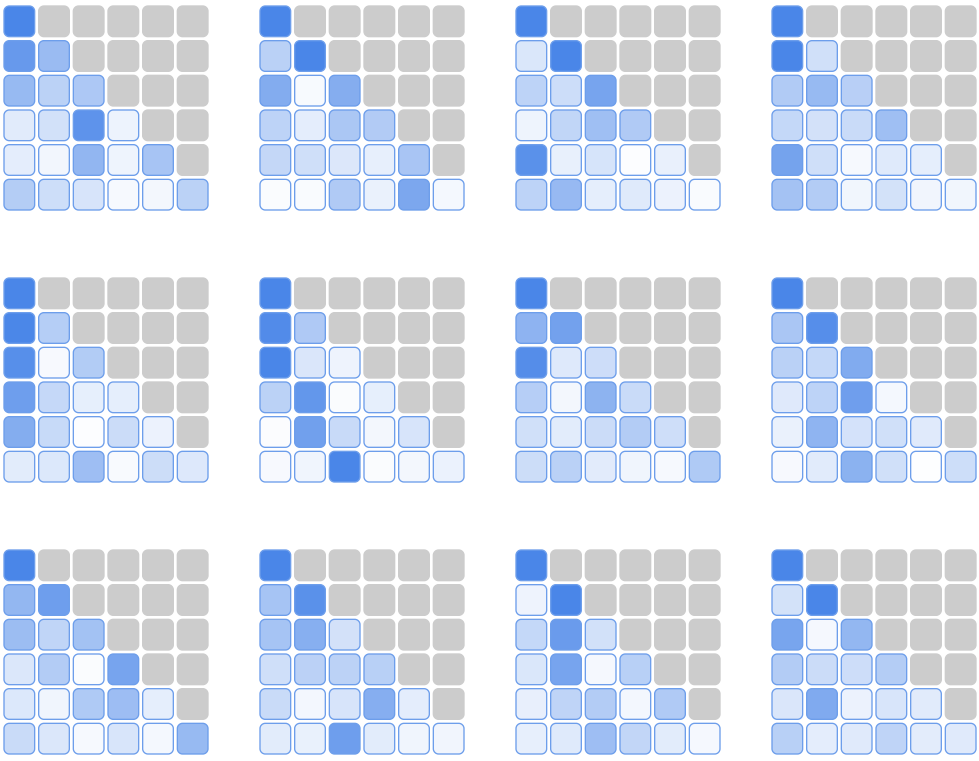


UNIT 01 · CHAPTER 9

Multi-Head Attention

CSE 10124 · Building ChatGPT

William Theisen · University of Notre Dame

Self-Attention, Again

In Self-Attention we gave every token a way to look straight back at the tokens before it. Each token's embedding is multiplied by three weight matrices to make a **query**, a **key** and a **value**, every query is compared with every key, and a softmax turns the scores into attention weights:

one row of context per token

[9.1] $$\text{Context} = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

the context is added back to the embeddings

[9.2] $$\mathcal{E}_{CA} = \mathcal{E} + \text{Context}$$

Our running example is the line Antony keeps repeating in *Julius Caesar*, "so are they all, all honorable", which we'll split into `6` word tokens (leaving out the comma). At the size of GPT-2 small, every token's embedding has `768` numbers, so $\mathcal{E}$ is `<6, 768>`. With W_Q and W_K of `<768, 64>`, Q and K are `<6, 64>`, and the attention weights are `<6, 6>`, one row per token.

Look at a single row of those weights, say the row for "honorable". It's `6` numbers that add up to `1`, and the context for "honorable" is the values of the `6` tokens averaged with those weights. **One row, one weighted average.** Whether one average is enough depends on what "honorable" needs to know.

One Weighted Average

The next word in the play is "men". To predict it after "so are they all, all honorable", the model has to know two quite different things.

It needs to know **who is being described**. That's "they", three tokens back, and "they" is plural, which is the only reason the answer is "men" and not "man". It also needs to know **what kind of word it's sitting on**. "Honorable" is a describing word, and a describing word is usually followed by the thing it describes.

Those are two different questions, and their answers are in two different places, "they" for the first and "honorable" itself for the second. Our single row of attention weights has to cover both. The weights add up to `1`, so every bit of attention spent on "they" is attention not spent on "honorable".

Unfortunately one row of weights can only strike one balance between the two. The easiest way to see what that costs is with a self-attention small enough to work out by hand.

Attention by Hand

Here's a self-attention small enough to do with a pencil. Each token gets just **4** numbers, and we picked them ourselves so each one means something: does the word say *who* we're talking about, is it *plural*, is it a *describing* word, or is it something *other*.

token	who	plural	describes	other
so	0	0	0	1
are	0	1	0	1
they	1	1	0	0
all (twice)	0	1	0	1
honorable	0	0	1	0

We pick W_Q and W_K ourselves too. A describing word's query asks both questions at once, and a key answers the first if its token says who, the second if its token describes:

each <4, 2>

$$W_Q = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 2 & 2 \\ 0 & 0 \end{bmatrix} \qquad W_K = \begin{bmatrix} 2 & 0 \\ 0 & 0 \\ 0 & 2 \\ 0 & 0 \end{bmatrix}$$

Half and Half

So "honorable" gets the query [2, 2], "they" gets the key [2, 0], "honorable" gets the key [0, 2], and every other token gets [0, 0]. The query's dot product with each key is 4 for "they" and "honorable" and 0 for the rest. Divided by √2 that's 2.83, and the softmax turns the six scores into 0.447 for "they", 0.447 for "honorable" and 0.026 for each of the other four.

To finish the attention we need values. We'll keep it simple and let each token's value be its embedding (W_V is the identity), so the context for "honorable" is 0.447 of "they", 0.447 of "honorable" and a little of everything else:

$$\underset{\text{who, plural, describes, other}}{\text{Context}_{\text{honorable}}} = \begin{bmatrix} 0.447 & 0.526 & 0.447 & 0.106 \end{bmatrix}$$

The query asked two questions and got back **half of each answer**. "Who" comes through at 0.447 and "describes" at 0.447, neither of them clearly.

Bigger numbers in W_Q and W_K wouldn't help much. The scores for "they" and "honorable" are tied, so the softmax splits the attention between them however sharp it is, and if we tilt the query towards one of them the other answer just fades further. As long as both questions share one row of weights, **being surer about "they" means saying less about "honorable"**.

Two Heads

The fix is to stop asking both questions in one attention. Give the first column of Q and K to one attention and the second column to another, and let each do its own softmax. Each now compares one number with one number (so d_k is `1` and there's nothing to divide by).

In the first, only "they" has a key, so "honorable" scores `4` against "they" and `0` against everything else, and the softmax gives "they" `0.916`. The second does the same for "honorable" itself:

	so	are	they	all	all	honorable
one head	0.026	0.026	**0.447**	0.026	0.026	**0.447**
head 1	0.017	0.017	**0.916**	0.017	0.017	0.017
head 2	0.017	0.017	0.017	0.017	0.017	**0.916**

Each of these is called an **attention head**, and running several heads side by side on the same tokens is **multi-head attention**. **Each head gets its own row of weights**, so neither has to give anything up for the other.

Two Heads in Code

Here's our hand-made example in PyTorch (`WQ` is W_Q, `WK` is W_K). The one head uses both columns, each of the two heads uses one:

```
import torch
# who, plural, describes, other
E = torch.tensor([[0., 0, 0, 1],     # so
                  [0., 1, 0, 1],     # are
                  [1., 1, 0, 0],     # they
                  [0., 1, 0, 1],     # all
                  [0., 1, 0, 1],     # all
                  [0., 0, 1, 0]])    # honorable
WQ = torch.tensor([[0., 0], [0, 0], [2, 2], [0, 0]])
WK = torch.tensor([[2., 0], [0, 0], [0, 2], [0, 0]])
Q, K = E @ WQ, E @ WK
one = torch.softmax(Q @ K.T / 2 ** 0.5, dim=-1)
print("one head:", one[5])
for h in range(2):
    q, k = Q[:, h:h+1], K[:, h:h+1]   # head h's column
    a = torch.softmax(q @ k.T, dim=-1)
    print("head", h + 1, a[5])
```

OUTPUT

```
one head: tensor([0.0264, 0.0264, 0.4471, 0.0264, 0.0264, 0.4471])
head 1 tensor([0.0168, 0.0168, 0.9161, 0.0168, 0.0168, 0.0168])
head 2 tensor([0.0168, 0.0168, 0.0168, 0.0168, 0.0168, 0.9161])
```

Each head also gets its own slice of the values: if head 1 takes the first two value columns (who, plural) and head 2 the last two (describes, other), the two contexts for "honorable" are [`0.916`, `0.966`] and [`0.916`, `0.067`]. Side by side that's [`0.916`, `0.966`, `0.916`, `0.067`], where the one head gave us [`0.447`, `0.526`, `0.447`, `0.106`], with **both answers at almost full strength**.

Different Questions

Our two questions were about grammar, but a head can ask anything that helps predict the next token. Take "Alice gave Bob a book." A token near the end of that sentence might want to know **who gave** (Alice), **who received** (Bob) and **what was given** (the book). Those are three different lookups, each with its own query, its own idea of a matching key and its own information to bring back, exactly what a head has.

Of course nobody tells a real model what its heads should ask. We picked our two questions by hand, but in a trained model every head's W_Q, W_K and W_V are just more weights, **learned by gradient descent like every other weight**. The heads end up asking whatever questions help predict the next token.

It turns out some heads do settle on recognizable jobs. When Clark and colleagues looked inside BERT in `2019`, they found a head where direct objects attend to their verbs, one where determiners attend to their nouns and one where pronouns attend to the earlier words they refer to. Most heads are much harder to put a name to.

Heads also aren't the only way to ask more than one question. **More heads** ask several questions of the same embeddings at once, and **more layers** let a later layer ask new questions using the answers an earlier one brought back.

Combining the Heads

Each head produces its own context, so we need some way to turn several contexts back into one. The first step is to put them side by side, **concatenating** them (written ⧺). With two heads whose values have 384 numbers each, our six tokens get two $\langle 6, 384 \rangle$ contexts, and side by side that's $\langle 6, 768 \rangle$, the size of $\mathcal{E}$ again.

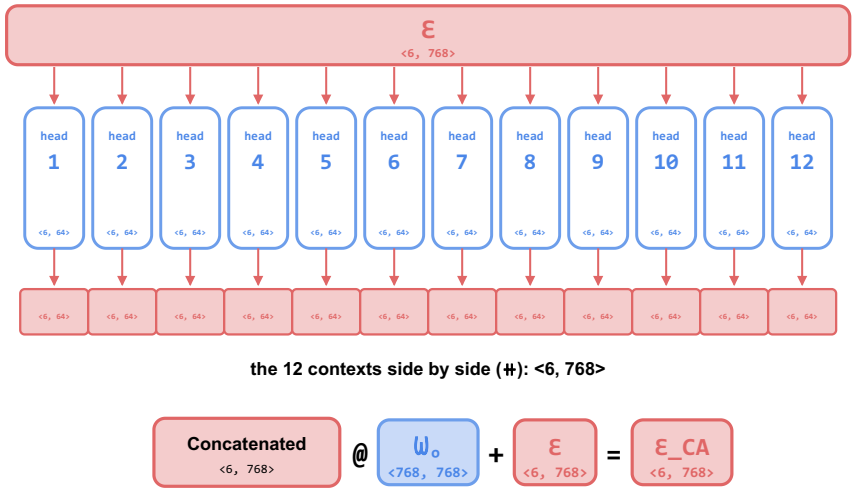
Concatenating alone leaves each head's answer in its own slot, though: the first 384 numbers only ever come from head 1 and the last 384 from head 2. So the concatenated context is multiplied by one more weight matrix, the **output weights** W_O $\langle 768, 768 \rangle$, which **mixes the heads' answers** so every one of the 768 numbers we add back to $\mathcal{E}$ can use all of them:

```
Missing $ inserted.
1.5 ...\,\red{K_i}^{T}}{\sqrt{d_k}}}\,\green{V_i}}
```

With a single head we didn't need W_O, since W_V was already $\langle 768, 768 \rangle$ and the context came out the right size to add to $\mathcal{E}$ straight away.

Head Size

GPT-2 small has 12 heads in every attention layer, and it gets their size by sharing its 768 numbers out evenly: 768 / 12 is 64, so every head's W_Q, W_K and W_V are <768, 64>. The twelve contexts of <6, 64> side by side are <6, 768> again, ready for W_O:



Splitting the numbers this way means **twelve heads cost the same as one**. One head's W_Q holds 768 × 64 = 49,152 weights, twelve of them hold 589,824, and that's exactly 768 × 768, the size of a single head's W_Q if it used all 768 numbers. The same goes for W_K and W_V.

The original transformer did the same with 8 heads of 64 for 512 numbers. irishQwen sets its head size separately, 16 heads of 128 for its 1,024 numbers, which is perfectly allowed: W_O just has to be <2048, 1024> to bring the result back to size.

Multi-Head Attention in Code

The most direct way to write multi-head attention is a loop, one head at a time, each with its own three weight matrices. We use random weights and embeddings at GPT-2 small's size, and the mask from Self-Attention so no token looks at the tokens after it:

```
import torch
torch.manual_seed(0)
T, d, H = 6, 768, 12
dk = d // H            # 64 numbers per head
E = torch.randn(T, d)  # E: our 6 tokens
mask = torch.triu(torch.ones(T, T), diagonal=1).bool()

heads = []
for h in range(H):
    WQ = torch.randn(d, dk) / d ** 0.5  # this head's
    WK = torch.randn(d, dk) / d ** 0.5  # own weights
    WV = torch.randn(d, dk) / d ** 0.5
    Q, K, V = E @ WQ, E @ WK, E @ WV  # <6, 64> each
    S = (Q @ K.T / dk ** 0.5).masked_fill(mask, -1e9)
    A = torch.softmax(S, dim=-1)        # <6, 6>
    heads.append(A @ V)                 # <6, 64>

ctx = torch.cat(heads, dim=1)  # <6, 768>
WO = torch.randn(d, d) / d ** 0.5
E_CA = E + ctx @ WO
print(heads[0].shape, ctx.shape, E_CA.shape)
```

OUTPUT

```
torch.Size([6, 64]) torch.Size([6, 768]) torch.Size([6, 768])
```

Every head's context is `<6, 64>`, `torch.cat` puts the twelve side by side, and after W_O the result is the same shape as E, so the two can be added.

Logical Partitioning

Twelve small matrix multiplications in a row is slow on a GPU, which would much rather do one big one. Twelve `<768, 64>` matrices placed side by side make one `<768, 768>` matrix, and multiplying E by the big matrix gives every head's queries at once, in `64`-column blocks. Head 1 owns columns `0` to `63`, head 2 owns `64` to `127`, and so on. Taking head 3's block gives the same numbers as multiplying by head 3's own columns:

```
import torch
torch.manual_seed(0)
T, d, H = 6, 768, 12
dk = d // H
E = torch.randn(T, d)
WQ = torch.randn(d, d) / d ** 0.5  # 12 heads side by side
Q = E @ WQ                 # <6, 768>, every head
Q3 = E @ WQ[:, 192:256]  # head 3's 64 columns
print(torch.allclose(Q[:, 192:256], Q3, atol=1e-5))
```

OUTPUT

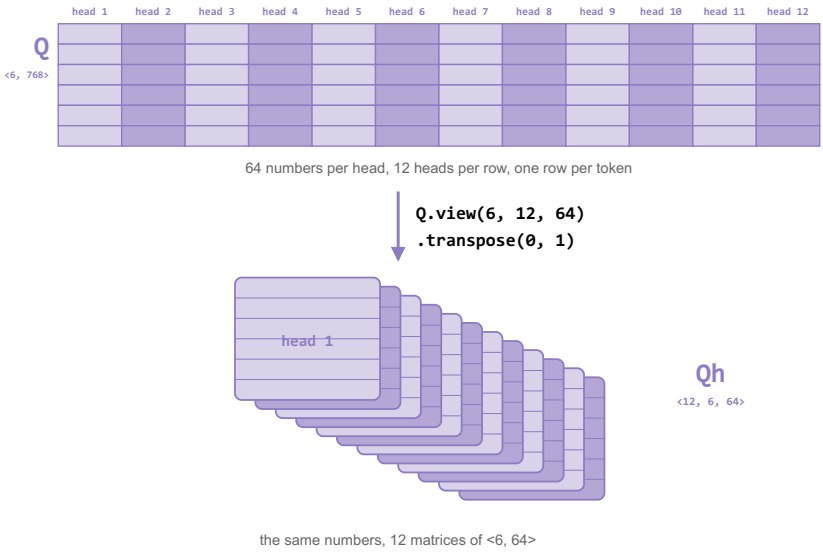
```
tensor(True)
```

This is called **logical partitioning**: **one weight matrix and one multiplication** make Q (and K, and V) for every head, and each head just reads its own section. The heads only exist in how we slice the result.

Our hand-made example was built this way all along. Its W_Q and W_K were each one `<4, 2>` matrix, and the two heads were their two columns.

Splitting the Heads

To give each head its section without a loop, we reshape. `Q.view(6, 12, 64)` reads each token's **768** numbers as **12** blocks of **64**, and `transpose(0, 1)` swaps the first two dimensions, so the heads come first. Nothing is computed, the same numbers are just grouped differently:



Now `Qh` is `<12, 6, 64>`, twelve `<6, 64>` matrices in a stack, and `@` works on stacks: `Qh @ Kh.transpose(1, 2)` multiplies head 1's queries by head 1's keys, head 2's by head 2's, and so on, giving `<12, 6, 6>`, **twelve sets of attention weights in one multiplication**.

irishQwen does exactly this in `_split_heads`, with one more dimension in front for a batch of prompts and `permute` doing the swap.

All Twelve Heads at Once

Here's multi-head attention with logical partitioning. At the end the contexts are swapped back and reshaped to `<6, 768>`, which concatenates them for free:

```python
import torch
torch.manual_seed(0)
T, d, H = 6, 768, 12
dk = d // H
E = torch.randn(T, d)
WQ, WK, WV, WO = [torch.randn(d, d) / d ** 0.5
                  for _ in range(4)]
mask = torch.triu(torch.ones(T, T), diagonal=1).bool()

Q, K, V = E @ WQ, E @ WK, E @ WV       # <6, 768>
Qh = Q.view(T, H, dk).transpose(0, 1)  # <12, 6, 64>
Kh = K.view(T, H, dk).transpose(0, 1)
Vh = V.view(T, H, dk).transpose(0, 1)

S = Qh @ Kh.transpose(1, 2) / dk ** 0.5 # <12, 6, 6>
A = torch.softmax(S.masked_fill(mask, -1e9), dim=-1)
ctx = (A @ Vh).transpose(0, 1).reshape(T, d)
E_CA = E + ctx @ WO
print(Qh.shape, A.shape, ctx.shape, E_CA.shape)
print(A[0])
```

OUTPUT

```
torch.Size([12, 6, 64]) torch.Size([12, 6, 6]) torch.Size([6, 768]) torch
tensor([[1.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
        [0.6002, 0.3998, 0.0000, 0.0000, 0.0000, 0.0000],
        [0.4089, 0.2674, 0.3237, 0.0000, 0.0000, 0.0000],
        [0.1179, 0.1757, 0.6358, 0.0706, 0.0000, 0.0000],
        [0.1048, 0.0509, 0.4291, 0.0675, 0.3477, 0.0000],
        [0.2979, 0.1970, 0.1567, 0.0336, 0.0479, 0.2669]])
```

The last six lines are head 1's attention weights: each row adds up to `1` and the masked weights are `0`.

Checking the Split

It's worth checking that reshaping didn't scramble anything. If logical partitioning really is the same as separate heads, then columns `192` to `255` of the context should match head 3 worked out entirely on its own, from its own `64` columns of Q, K and V:

```
import torch
torch.manual_seed(0)
T, d, H = 6, 768, 12
dk = d // H
E = torch.randn(T, d)
WQ, WK, WV = [torch.randn(d, d) / d ** 0.5
              for _ in range(3)]
mask = torch.triu(torch.ones(T, T), diagonal=1).bool()
Q, K, V = E @ WQ, E @ WK, E @ WV
split = lambda X: X.view(T, H, dk).transpose(0, 1)

S = split(Q) @ split(K).transpose(1, 2) / dk ** 0.5
A = torch.softmax(S.masked_fill(mask, -1e9), dim=-1)
ctx = (A @ split(V)).transpose(0, 1).reshape(T, d)

# head 3 on its own, from its 64 columns
q, k, v = Q[:, 192:256], K[:, 192:256], V[:, 192:256]
s = (q @ k.T / dk ** 0.5).masked_fill(mask, -1e9)
head3 = torch.softmax(s, dim=-1) @ v
print(torch.allclose(ctx[:, 192:256], head3, atol=1e-5))
```

OUTPUT

```
tensor(True)
```

They match (`allclose` allows for the last digit or so of rounding). So the twelve heads in the loop and the twelve heads in one big multiplication are the same twelve heads, **only the bookkeeping is different**. That's how attention layers are written in practice.

Generating Text

So far we've run attention once over a fixed prompt. A language model generating text runs it over and over. Given "so are they all, all honorable", it predicts the next token, say "men", adds it to the end, and runs again on all `7` tokens to predict the token after that. Generating one token at a time, each one fed back in, is called **auto-regressive** generation.

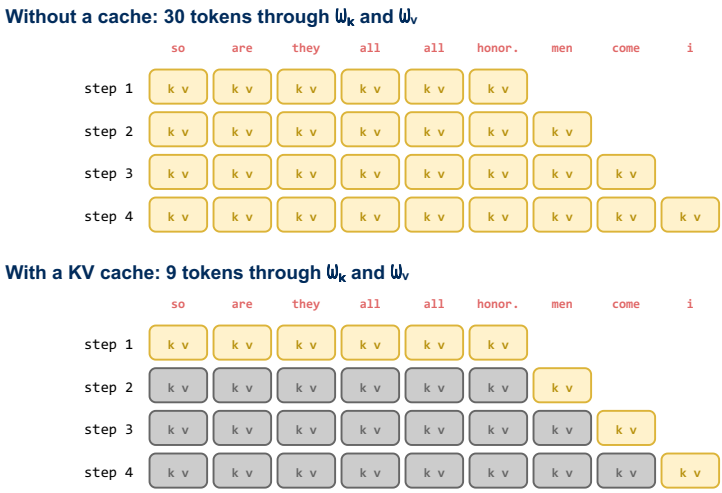
Done exactly like that, the model does a lot of work twice. At every step every token goes through W_Q, W_K and W_V again, in every head of every layer. With our `6`-token prompt and `100` new tokens, the steps run `6`, `7`, `8` … `105` tokens, `5,550` in all, when each token only really needed to go through once, `105` times in all.

Most of that is the same arithmetic repeated, but only if the old numbers really don't change when a new token arrives. Whether they do comes down to the mask.

What Changes and What Doesn't

Because of the mask, a token only ever attends to itself and the tokens before it. When "men" is added, nothing about "so", "are", "they", "all", "all" or "honorable" changes: they couldn't see "men" before it arrived and they still can't. Their contexts are the same, and so are their keys and values, **in every head of every layer**.

The only new work is for the new token. "men" needs its own query, key and value, and its query has to be compared with every key so far. Pictured below are the keys and values worked out at each of the first four steps, without and with keeping the old ones:



Notice also that the old queries are never needed again. Only the newest token's row of attention weights is used to predict the next token, and that row needs **one query but every key and value**.

The KV Cache

So we keep them. A **KV cache** stores the keys and values of every token so far, so they can be reused instead of worked out again. At each step only the new token goes through W_Q, W_K and W_V, its key and value are added to the cache, and its query is compared with every cached key.

In multi-head attention **every head has its own cache**, since every head has its own keys and values. In GPT-2 small, after our `6` tokens, each head's cache holds a `<6, 64>` matrix of keys and another of values, and every layer has twelve heads. With logical partitioning these are just slices of one `<12, 6, 64>` tensor of keys and one of values per layer.

For our `100` new tokens, that's `105` tokens through W_K and W_V instead of `5,550`. Unfortunately the comparisons don't shrink the same way: each new query is still compared with every key in the cache, so **a longer conversation still means more work per token**, just much less than before.

A KV Cache in Code

Here's one attention layer generating "men" with a cache. The prompt's keys and values go in first, then the new token's are added with `torch.cat`, and only the new token's query is made. The embeddings are random, so it could be any token:

```python
import torch
torch.manual_seed(0)
d, H = 768, 12
dk = d // H
WQ, WK, WV = [torch.randn(d, d) / d ** 0.5
              for _ in range(3)]
split = lambda X: X.view(len(X), H, dk).transpose(0, 1)
E = torch.randn(7, d)  # 6 prompt tokens, then "men"

# prefill: the whole prompt at once fills the cache
K_cache = split(E[:6] @ WK)  # <12, 6, 64>
V_cache = split(E[:6] @ WV)

# decode: only the new token goes through the weights
e = E[6:]          # <1, 768>
K_cache = torch.cat([K_cache, split(e @ WK)], dim=1)
V_cache = torch.cat([V_cache, split(e @ WV)], dim=1)
q = split(e @ WQ)  # <12, 1, 64>
S = q @ K_cache.transpose(1, 2) / dk ** 0.5
A = torch.softmax(S, dim=-1)
new = A @ V_cache  # <12, 1, 64>
print(K_cache.shape, new.shape)
```

OUTPUT

```
torch.Size([12, 7, 64]) torch.Size([12, 1, 64])
```

The cache now holds `7` tokens' keys and values for each of the `12` heads, and the new token's context `new` has one row per head. Only one token went through W_Q, W_K and W_V to get it.

Checking the Cache

To be sure the cache gives the right answer, we can work out the same step the long way: all 7 tokens through the weights again, with the mask, exactly as if there were no cache. This cell carries on from the one before, so the earlier cell runs first and prints its line again:

```
# the long way: all 7 tokens again, with the mask
mask = torch.triu(torch.ones(7, 7), diagonal=1).bool()
Q, K, V = split(E @ WQ), split(E @ WK), split(E @ WV)
S = Q @ K.transpose(1, 2) / dk ** 0.5
S = S.masked_fill(mask, -1e9)
full = torch.softmax(S, dim=-1) @ V  # <12, 7, 64>
print(torch.allclose(new[:, 0], full[:, 6], atol=1e-5))
```

OUTPUT

```
torch.Size([12, 7, 64]) torch.Size([12, 1, 64])
tensor(True)
```

The new token's context from the cache matches the last row of the full calculation. **Caching changes how much we compute, not what we compute.** There's no mask in the cached version because there's nothing to hide, the cache only holds the past.

Prefill and Decode

With a cache, answering a prompt happens in two quite different phases. First the model reads the whole prompt. Every prompt token is already known, so they can all go through the layers at once, in parallel, the way we've been running attention all chapter. This is called **prefill**, and it fills the KV cache.

Then the model writes. Each new token depends on the one before, so they come out one at a time, each going through the layers alone and attending to the cache. This is called **decode**. Prefill is **a few big multiplications**, decode is **many small ones**, and every new message you send is prefilled before the model starts decoding its reply.

irishQwen has a method for each, `prefill` and `decode_step`, and its `chat` method calls the first once and the second for every new token. The comments in its attention layer put it simply enough: "Same weights, just reusing work!"

The Size of the Cache

The cache isn't free, it has to sit in memory for the whole conversation. Every token needs a key and a value of `768` numbers (twelve heads of `64`) in each of GPT-2 small's `12` layers, which is `18,432` numbers per token. At GPT-2's limit of `1,024` tokens that's `18,874,368` numbers for a single conversation.

irishQwen, with Qwen3-0.6B's shapes, is bigger in every direction: `28` layers, `16` heads of `128` and room for `4,096` tokens. If every head kept its own keys and values, a full cache would hold `469,762,048` numbers, about `1.9` GB at the `4` bytes irishQwen uses per number, **for one conversation**, on top of the model itself.

It turns out irishQwen keeps only `8` heads' worth of keys and values for its `16` heads of queries, which halves that. How heads can share keys and values, and what else people have changed about attention to make it cheaper, is where the next chapter, Attention Variants, picks up.